

■ WISSENSMANAGEMENT IN 2026



Doctus.

AI Driven Wissensmanagement. On Premise.

Out of the box.

Drei Fragen, die den Startpunkt festlegen.

01 · PROBLEM

Welches Problem hat der Kunde und lösen wir?

Das lösen wir heute schon

- Code-Archäologie vor jeder Migration
- Wissensverlust bei Pensionierung
- Audit- & Compliance-Recherche von Hand
- Monatelanges Onboarding
- Undokumentierter Bestand
- Unbelegbare Aufwandsschätzung
- Wissen verstreut über Code, Confluence, Jira

02 · ANGEBOT

Produkt, Projekte, sonstige Services?

So könnte es angeboten werden

- SaaS
- Modernisierung
- Serviceprojekte

03 · KUNDEN

Wie sieht der Kunde aus?

Diese Zielgruppen kommen zuerst infrage

- Banken & Sparkassen
- Versicherungen
- Öffentlicher Sektor
- Industrie & Logistik

Das Wissen sitzt in Köpfen, nicht in Systemen.

Große COBOL-Bestände laufen seit Jahrzehnten stabil — ihr Verständnis hängt an wenigen, langjährigen Personen. Dokumentation existiert, ist aber über Code, Confluence und Jira verstreut und selten aktuell. Das ist kein Softwareproblem, es ist ein Wissensproblem — und es hat erstmals eine Uhr.

> 800 Mrd.

Zeilen COBOL laufen produktiv — und es kommen rund 5 Mrd. pro Jahr dazu

Ø 55 Jahre

Durchschnittsalter der COBOL-Entwickler; rund 10 % gehen jedes Jahr in den Ruhestand

> 85 %

der Hochschulen haben COBOL aus dem Curriculum gestrichen — es kommt niemand nach

Der Bestand wächst, die Zahl der Menschen, die ihn erklären können, sinkt. Genau diese Schere ist der Markt — und sie schließt sich nicht von selbst.

Quellen: Micro Focus / Open Mainframe Project (COBOL-Bestand, 2022–2025) · IBM (jährlicher Zuwachs) · Computerworld / Open Mainframe Project Demografie-Erhebungen.

Zwei Wege in denselben Markt — und sie kommen auf dieselbe Zahl.

TAM	Zwei Referenzmärkte, nicht einer WEG A · PROGRAMM 8,4 → 13,3 Mrd. \$ Mainframe-Modernisierung weltweit, Software und Services. CAGR 9,7 %. Projektgeschäft, hoher Anteil Mensch. WEG B · PRODUKT 7,7 → 22,2 Mrd. \$ KI-Werkzeuge für Softwareentwicklung, Abo-Modell. CAGR 23,8 %. Wächst 2,5-mal so schnell, skaliert ohne Kopfzahl. 8,4 / 7,7 Mrd. \$ · 2025 Programm / Produkt
SAM	Deutschland — zweimal gerechnet Beide Wege getrennt hergeleitet. Dass sie sich treffen, ist der Beleg — nicht die einzelne Zahl. TOP-DOWN · AUS DEM MARKT 40–60 Mio. € Deutschland 436 Mio. \$ (2025 → 665 Mio. \$ 2030), rund 21 % Europas. Davon die Verstehens-Phase: Annahme 10–15 % des Programmvolumens. BOTTOM-UP · AUS DEN ACCOUNTS 15–60 Mio. € Rund 250–400 Organisationen in Deutschland mit eigenem geschäftskritischem COBOL-Bestand, bei 60–150 T€ wiederkehrend pro Jahr. 40–60 Mio. € Deutschland, pro Jahr Überschneidung beider Wege
SOM	Was wir in drei Jahren realistisch holen Qualifizierte Accounts im Fujitsu-Bestand mit geschäftskritischem COBOL → 6–10 Kunden in drei Jahren: Discovery-Pilot 30–60 T€ als Einstieg, danach 80–150 T€ pro Jahr aus Plattform plus Nutzern, nicht aus Seats allein — COBOL-Teams sind mit 10–60 Personen dafür zu klein. 0,8–1,8 Mio. € wiederkehrend ab Jahr 3

Was hier Annahme ist: Accountzahl und der 10–15-%-Anteil der Verstehens-Phase — beides geschätzt, nicht erhoben. Der interne Einsatz bei Fujitsu steht bewusst nicht im Trichter: er ist ein Margenhebel auf bestehendes Geschäft, kein zusätzlicher Markt.

Quellen: MarketsandMarkets, „Mainframe Modernization Market“ (2025): global 8,39 → 13,34 Mrd. \$, Deutschland 436,33 → 664,84 Mio. \$ · Grand View Research, „AI Code Tools Market“ (2025): 7,65 → 22,2 Mrd. \$, CAGR 23,8 % · Bitkom (07/2026): ITK-Markt Deutschland = rund 4,3 % des Weltmarkts · Bundesbank (2025): 1.329 Kreditinstitute · BaFin: rund 500 Versicherer. Umrechnung 1,08 \$/€. SAM-Anteil, Accountzahl und SOM sind eigene Annahmen.

Vier Branchen — vier Use Cases

Banken & Sparkassen Kernbank, Zahlungsverkehr, Meldewesen. Größtes COBOL-Volumen, höchste Regulierungsdichte. Der Account ist selten das einzelne Institut, sondern die Rechenzentrale, in der die Entwicklung gebündelt ist — wenige Türen, dahinter große Bestände. ----- Anlass: DORA, Meldewesen, Kernbank-Ablösung	Versicherungen Bestandsführung mit Vertragslaufzeiten über Jahrzehnte — Fachlogik von 1985 ist heute noch bindend. ----- Anlass: Bestandsmigration, Tarifwerk	Öffentlicher Sektor Steuer, Sozial- und Rentenverfahren. Ablösung politisch getrieben, Ausfall nicht verhandelbar. ----- Anlass: Verfahrensablösung, Fachkräftemangel	Industrie & Logistik Disposition, Abrechnung, ERP-Vorläufer — oft undokumentiert gewachsen, ohne Herstellersupport. ----- Anlass: ERP-Konsolidierung, Personalabgang
--	---	---	--

Was alle vier eint: jahrzehntelang gewachsene, geschäftskritische COBOL-Bestände, deren Fachlogik kaum dokumentiert ist und nur noch in wenigen Köpfen liegt — bei **Änderungsdruck von außen, dem man sich nicht entziehen kann.**

01 Modernisierungsprojekte Vor Migration oder Ablösung steht Verstehen — heute Wochen bis Monate manueller Code-Archäologie, mit einer Aufwandsschätzung, die niemand belegen kann.	02 Wissensverlust Pensionierung oder Wechsel reißt Systemverständnis heraus, das nirgends sonst dokumentiert war — und das nachträglich niemand rekonstruiert.	03 Audit oder Compliance Wo wird welches Datenfeld verarbeitet? Ohne Aufrufgraph eine aufwändige Handrecherche — mit einem Ergebnis, das man nicht belegen kann.	04 Onboarding neuer Entwickler Einarbeitung in einen 30 Jahre gewachsenen Bestand dauert Monate. Genau die Zeit, die bei der Nachbesetzung fehlt.
--	---	---	--

Wir hatten diese Idee schon einmal. **Sie ist gescheitert.**

Das Doctus Projekt war ein Versuch, Wissen über Bestandssysteme hinweg verfügbar zu machen. Offiziell endete es nach dem ersten PoC — ohne weitere Beauftragung.

Was 1.0 richtig gesehen hat

- **Das Problem war real.** Der Bedarf, den 1.0 adressieren wollte, ist heute größer als damals — nicht kleiner.
- **Der Zugang war richtig.** Bestandssysteme erklärbar machen, statt sie blind abzulösen.
- **Die Domänenkenntnis ist geblieben.** Alles, was wir über COBOL-Bestände, Copybooks und Aufrufstrukturen gelernt haben, steckt heute in 2.0.
- **Das Timing.** Der Bestand altert, die Wissensträger gehen — das Zeitfenster war damals offen und ist es heute noch.

Warum es für mich gescheitert ist

- **Kein Rollout.** Der Kunde hat zwar weiteres Interesse, wünscht sich aber ein besseres Tool.
- **Fehlender Fokus.** Es gab ein Problem beim Kunden oder ein Modernisierungs-Portfolio-Tool — nicht beides.
- **Technologische Entwicklung.** Der massive Sprung der Möglichkeiten verzerrt, was möglich ist und was möglich war.

Vier Fragen entscheiden.

1.0 hat **alle vier** nicht beantwortet.

FRAGE 01 Welches Problem löse ich genau?	FRAGE 02 Welche Technologie verwende ich dafür?	FRAGE 03 Wer arbeitet daran mit?	FRAGE 04 Was kostet das?
DER FEHLER Es war nie klar, welches Problem gelöst werden sollte. Ohne scharf umrissenes Problem wird jede Anforderung gleich wichtig — das Ergebnis ist eine Software, die niemand vermisst.	DER FEHLER Die Technologie wurde im Verlauf des Projekts alt. Was zu Beginn die richtige Wahl war, war bei jedem Zwischenstand ein Stück weiter überholt. Der Bau war langsamer als der Wandel drumherum.	DER FEHLER Das Team hat sich ständig fundamental geändert. Mit jedem Wechsel ging teilweise Kontext und getroffene Entscheidungen verloren.	DER FEHLER Die eingesetzten Ressourcen waren zu gering. Nicht falsch verplant — zu wenig, um je kritische Masse zu erreichen.

Doctus 2.0 ist der Versuch, dieselben vier Fragen **vorher zu beantworten.**

FRAGE 01 Welches Problem löse ich genau?	FRAGE 02 Welche Technologie verwende ich dafür?	FRAGE 03 Wer arbeitet daran mit?	FRAGE 04 Was kostet das?
DIE LÖSUNG Wissen wird zugänglich in der Softwareentwicklung : Fragen werden belegbar in der Situation beantwortet und unterstützen so im Prozess.	DIE LÖSUNG Ein modernes Frontend, ein austauschbares Sprachmodell . Retrieval-Augmented Generation mit Vektordatenbank auf Postgres bildet das Fundament; der eigentliche Wert steckt im Parser, im Graphen und in der Belegkette, die nicht mit dem Modell altern. Ausschließlich Open Source, kein Hersteller-Lock.	DIE LÖSUNG Eine durchgehende Verantwortung, klein gehalten . Kein wechselndes Projektteam, sondern ein stabiler Kern plus punktuelle Gegenleser aus Consulting und Sales — das ist die direkte Konsequenz aus 1.0.	DIE LÖSUNG Die Software ist bereits weit entwickelt. Das Produkt läuft und ist einsatzbereit. Was jetzt aussteht, ist kein Entwicklungsbudget, sondern der Rahmen für einen Piloten — benannt auf der Ressourcenfolie.

Kundenprobleme lösen, **by Design.**

PROBLEM „Wir können einer KI-Antwort über unser Kernsystem nicht trauen.“ Belegpflicht auf der Codezeile Jede Antwort zeigt die Quelle im Originalcode. Der Nutzer prüft in Sekunden statt zu glauben — und darf widersprechen.	PROBLEM „Unser Quellcode darf das Haus nicht verlassen.“ On-Premise, ausschließlich Open Source Läuft vollständig im Rechenzentrum des Kunden. MIT/BSD/Apache-2.0 — die Lizenzdiskussion, die solche Werkzeuge sonst monatelang aufhält, entfällt.
PROBLEM „Das Wissen liegt in Code, Confluence und Jira — nie an einem Ort.“ Code und Dokumentation vereint Git, Confluence, Jira, WebDAV, Ordner und Upload in einem durchsuchbaren Bestand. Kein Herstellerwerkzeug, das nur den Code sieht.	PROBLEM „Für jede Audit-Anfrage recherchiert jemand tagelang von Hand.“ Aufruf- und Wissensgraph mit Export CALL, PERFORM, GOTO, COPY visuell und exportierbar. Das Ergebnis bleibt beim Kunden — auch ohne uns.
PROBLEM „In zwei Jahren ist das eingesetzte Modell veraltet.“ — der Fehler aus 1.0 Modellunabhängige Architektur Das Sprachmodell ist ein austauschbarer Baustein, kein Fundament. Parser, Graph und Belegkette bleiben, wenn das Modell wechselt.	PROBLEM „Wir haben schon einmal in ein Konzept investiert, das nie lief.“ — der Fehler aus 1.0 Es läuft. Heute. Zehn von zehn Arbeitspaketen abgeschlossen, regressionsgetestet, deploybar. Kein Konzeptpapier, sondern ein Produkt, das man aufsetzen kann.

Warum das jetzt geht: **Erst seit Sprachmodelle lokal gut genug laufen, kann ein solches Werkzeug im Rechenzentrum eines regulierten Kunden stehen.** Das ist der eigentliche Auslöser — und zugleich das Zeitfenster, denn diese Machbarkeit ist nicht exklusiv.

■ BEVOR JEMAND ANDERS DIE FRAGE STELLT

Was Doctus ist — und was es **bewusst nicht ist.**

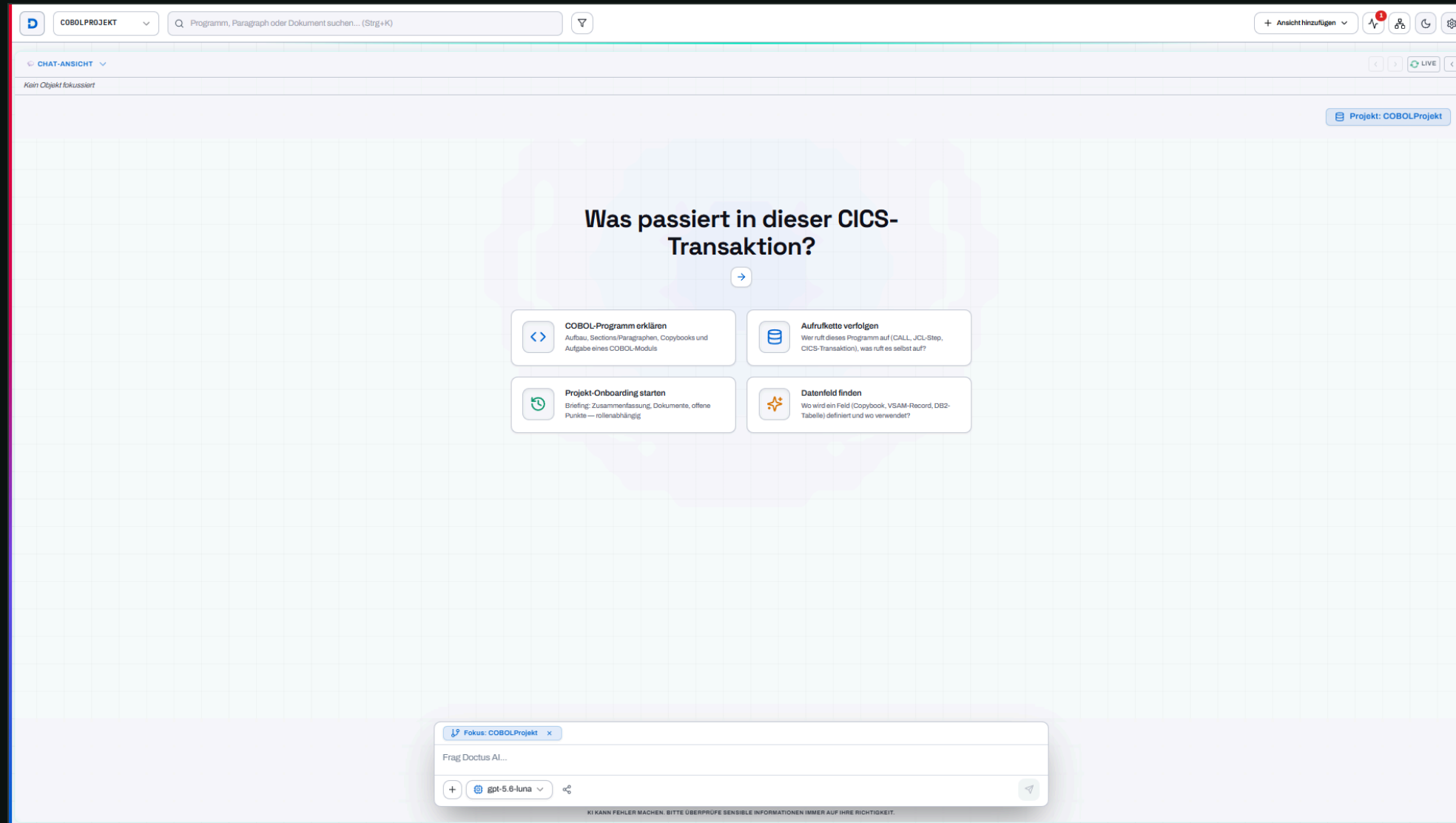
Was Doctus ist

- **Ein Wissensassistent für COBOL-Bestände.** Beantwortet Fragen zum Code mit Beleg auf der Zeile.
- **Ein Werkzeug zum Verstehen, nicht zum Verändern.** Macht Bestandswissen zugänglich, bevor irgendetwas angefasst wird.
- **On-Premise und Open Source.** Läuft im Rechenzentrum des Kunden, ohne Fremdlizenz.
- **Ein fertiges, lauffähiges Produkt.** Kein Konzept — eine Software, die heute schon existiert.

Was Doctus nicht ist

- **Kein Migrations- oder Konvertierungswerkzeug.** Es macht COBOL verständlich, es macht daraus kein Java.
- **Kein Code-Generator.** Es schreibt keinen Produktivcode und ändert nichts am Bestand.
- **Keine fachliche Wahrheitsgarantie.** Antworten kommen mit Quelle und Unsicherheitshinweis; die Bewertung bleibt bei einem Menschen.
- **Kein Ersatz für Fachexperten.** Es ist deren Hebel — das ist auch die ehrlichere Verkaufsgeschichte.

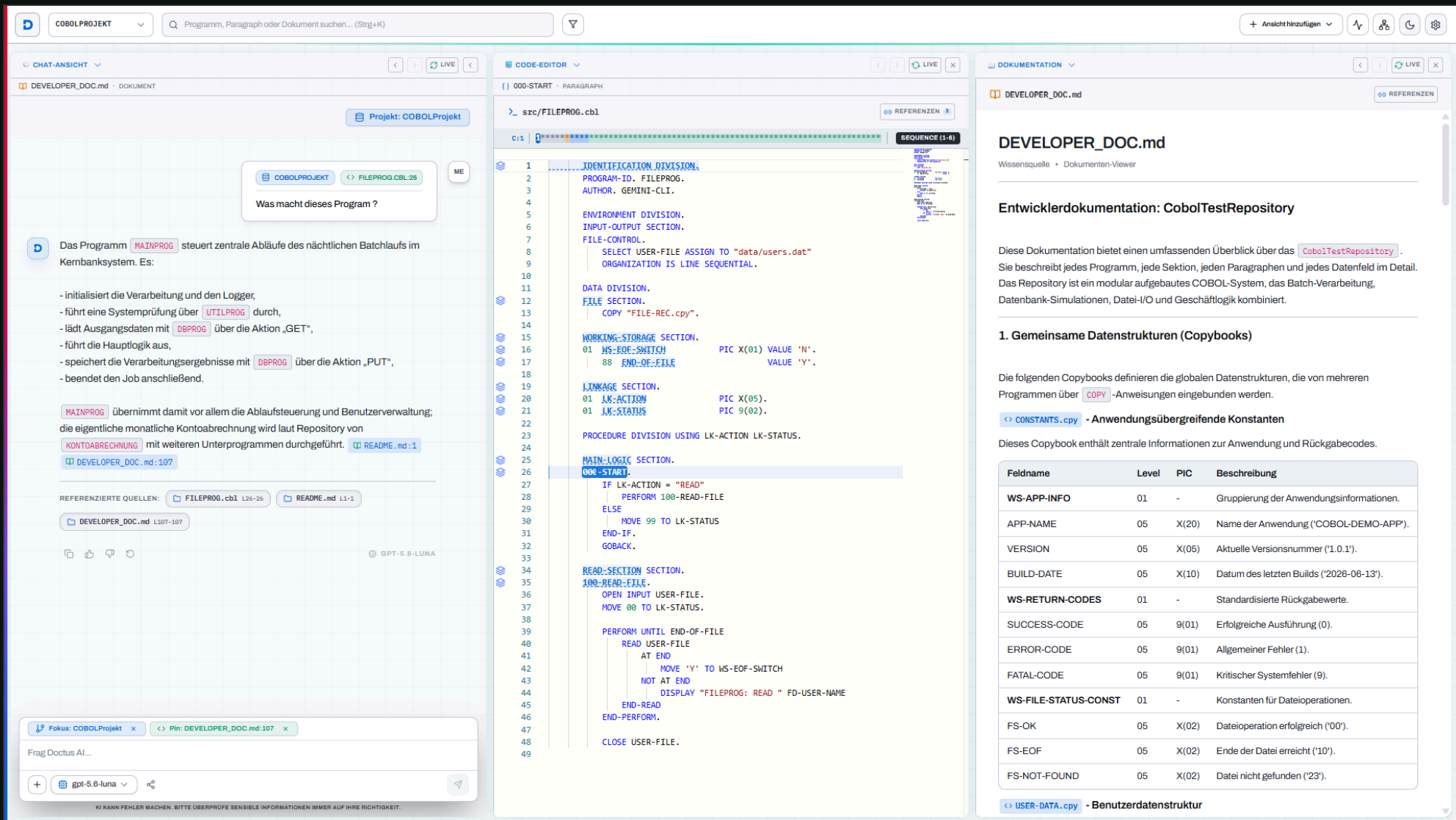
Initial: neuer Chat



Chat und Wissensgraph nebeneinander

The screenshot displays the Doctus AI interface, which is split into two main panels. The left panel, titled 'CHAT-ANSICHT', shows a chat window with a sidebar on the far left containing a 'VERLAUF' (History) section with messages like 'Was steht in der README?', 'Was macht dieses Program?', 'Was macht FIN-DATA.cpy?', 'Was ist das für ein Projekt?', and 'hallo'. The chat area itself has a header 'Projekt: COBOLProjekt' and a large question 'Wo wird auf diese DB2-Tabelle zugegriffen?'. Below the question are four interactive cards: 'COBOL-Programm erklären', 'Aufrufkette verfolgen', 'Projekt-Onboarding starten', and 'Datenfeld finden'. The right panel, titled 'WISSENSNETZ (GRAPH)', displays a complex knowledge graph with green nodes and orange lines representing relationships. A legend at the bottom left of the graph panel identifies the nodes as 'Git / Code-Elemente' and the links as 'Schlüsselwort'. The interface also includes a top navigation bar with a search bar and a bottom status bar with a disclaimer: 'KI KANN FEHLER MACHEN. BITTE ÜBERPRÜFE SENSIBLE INFORMATIONEN IMMER AUF IHRE RICHTIGKEIT.'

Drei Arbeitsfenster parallel



Sechs Dinge, **die heute schon existieren.**

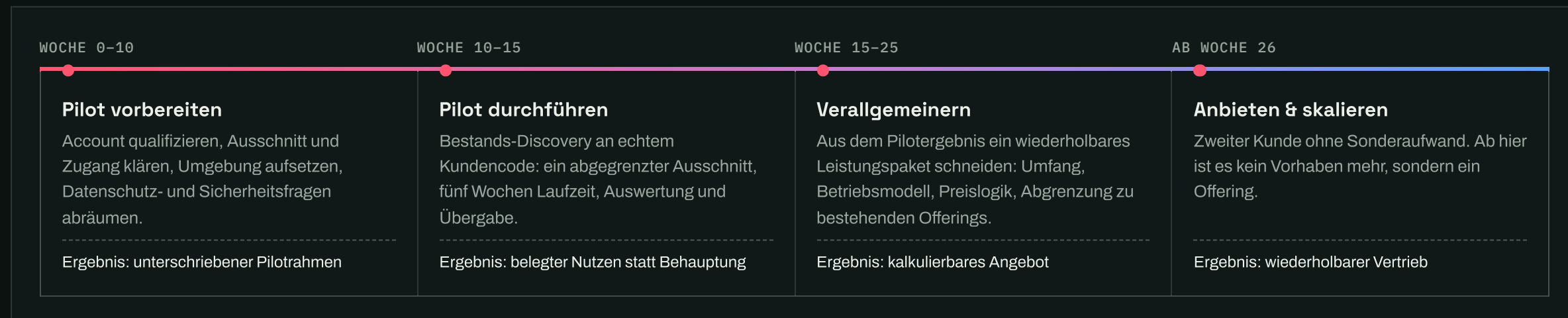
01 · PRODUKT Lauffähige Software Vollständig funktionsfähig: Parser, Wissens- und Aufrufgraph, Konnektoren, Chat mit Belegkette, Job-Center, Rollen- und Projektrechte.	02 · RECHTE Vollständig eigene Lösung Eigenentwicklung ohne durchzureichende Fremdlizenz. Ausschließlich MIT/BSD/Apache-2.0-Abhängigkeiten, dokumentiert im OSS-Clearing.	03 · BETRIEB Deploybares On-Prem-Paket Containerisiert, im Kundenrechenzentrum lauffähig, mit Deployment-Dokumentation. Cloud nur als ausdrückliches Opt-in.
04 · DOKUMENTATION Nachvollziehbar statt Kopfwissen Anforderungskatalog, Produkthandbuch, Architektur- und Datenmodell, Konnektor-Dokumentation, Mainframe-Kompatibilitätsmatrix — der 1.0-Fehler „Wissen geht mit dem Team“ ist hier bewusst ausgeschlossen.	05 · QUALITÄT Automatisierte Testabdeckung Regressionstests über Parser, Konnektoren, Rechte und Oberfläche — Änderungen sind belegbar, nicht behauptet.	06 · SICHERHEIT Sicherheitskonzept von Anfang an SSO/OIDC neben klassischem Passwort-Login, Passwort-Hashes nie in Logs oder Diagnose-Bundles, Cloud-LLMs nur als explizites Opt-in pro Kunde — On-Premise ist der Normalfall, nicht die Ausnahme.

- !

Offen: weitere Mainframe- und DB2-Konnektoren — für klassische Großkunden-Umgebungen ohne Git
- !

Offen: Compliance- und Paketierungsanforderungen — abhängig davon, welchen Angebotsweg wir wählen

Erster Pilot in gut drei Monaten. Angebotsreif **nach etwa einem halben Jahr.**



Was diesen Zeitplan verschiebt: Je größer die Diskrepanz zwischen dem, was gebraucht wird, und dem tatsächlichen Ist-Zustand des Bestands, desto mehr verschiebt sich der Zeitplan nach hinten.

Ein bis zwei FTE. Kein Programm, keine Gremienstruktur.

Die Kern-Entwicklung ist fast abgeschlossen. Was jetzt gebraucht wird, ist der Rahmen, um das Produkt an echten Kundenbestand zu bringen — und die Stabilität, an der wir bereits gescheitert sind.

ROLLE	AUFWAND	ANMERKUNG
Produkt & Entwicklung Weiterentwicklung, Konnektor-Ausbau, Pilotbegleitung — durchgehend dieselbe Person, ergänzt um einen zweiten Kollegen zur Hälfte	1,5 FTE	1,0 FTE besteht bereits. Die zusätzlichen 0,5 FTE sind der Wunsch nach einem zweiten Kollegen zur Entlastung.
Consulting-Gegenleser Fachliche Prüfung der Anwendungsfälle, Begleitung im Pilotkundengespräch	0,2 FTE	Punktuell, im Wesentlichen während des Piloten.
Sales / Account-Zugang Türöffner bei zwei bis drei qualifizierten Accounts	0,2 FTE	Kein Pitch — nur die drei Qualifizierungsfragen im ohnehin stattfindenden Gespräch.
Sonstiges Unvorhergesehenes, Abstimmung, Verwaltung	0,1 FTE	Puffer für Unvorhergesehenes.
Summe bis zum wiederholbaren Angebot Über die vier Roadmap-Schritte, rund sechs Monate	2,0 FTE	

Plus Sachmittel: Pilot-Infrastruktur (GPU-Kapazität, Testumgebung) im niedrigen fünfstelligen Bereich. **Kein Lizenzbudget** — der Stack ist vollständig Open Source.

COBOL ist der Einstieg, nicht die Grenze.

SPRACHEN	PL/I, Natural, RPG, JCL, Assembler — dieselben Bestände enthalten selten nur COBOL. Der Parser läuft seit dem letzten Umbau über eine Registry auf einer sprachneutralen Kernschicht; eine neue Sprache ist ein Parser plus Testkorpus, nicht ein neues Produkt.	VORBEREITET
QUELLSYSTEME	Mainframe-SCM und DB2 — Endavor, ChangeMan, PDS-Datasets. Git, Confluence, Jira, WebDAV und Dateiablagen sind angebunden; die Konnektorschicht ist modular, jeder weitere Konnektor öffnet ein Kundensegment, das heute nicht erreichbar ist.	NÄCHSTER AUSBAU
VERTRIEB	Vom Piloten zum wiederholbaren Paket. Der teuerste Teil sind die ersten Piloten, danach skaliert das Produkt mit relativ hohen Margen.	ROADMAP-SCHRITT 4
GEOGRAFIE	Keine Bindung an DACH. Das Produkt ist mehrsprachig bedienbar und nicht an lokale Regularien gebunden — die On-Premise-Auslegung ist gerade in stark regulierten Märkten ein Vorteil, kein Hindernis.	TECHNISCH OFFEN

Doctus verkürzt die Phase, die heute **am längsten** dauert.

Jedes Modernisierungsprogramm beginnt mit Verstehen — und genau diese Phase ist die, für die niemand eine belastbare Aufwandsschätzung abgeben kann. Sie ist der Grund für die meisten Nachträge.

PHASE 1 Discovery & Assessment Bestandsaufnahme, Abhängigkeiten, tote Programme, Schnittstellen nach außen. ----- Statt Wochen Handarbeit: Graph am ersten Tag	PHASE 2 Zielarchitektur & Schnitt Wo lässt sich der Bestand sinnvoll zerlegen? Welche Programme hängen wirklich zusammen? ----- Schnittkandidaten aus dem Aufrufgraph belegt	PHASE 3 Migration & Neubau Fachlogik verstehen, bevor sie neu geschrieben wird — mit Beleg, woher jede Regel stammt. ----- Spezifikation mit Herkunftsnachweis	PHASE 4 Nachweis & Abnahme Welche Regel des Altsystems ist wo im Neubau umgesetzt? Die Frage, die im Audit kommt. ----- Nachvollziehbarkeit alt ↔ neu
--	--	--	---

DER HEBEL	Doctus ersetzt kein Modernisierungsprojekt — es macht das eigene Angebot besser schätzbar . Das ist zugleich der günstigste Einstieg beim Kunden: eine abgegrenzte Discovery, die er ohnehin braucht. Und weil wir in allen Phasen eines Programms unterstützen können, endet der Nutzen nicht nach der Discovery.
-----------	---

Danke — Zeit für Fragen und Diskussionen.

Ohne diese Antworten steht Doctus 2.0 an derselben Stelle wie 1.0.

- | | |
|----|---|
| 01 | Gibt es Accounts, die die harten Kriterien (COBOL-Code, Softwareentwicklung, strenge Regulatorik) voraussichtlich erfüllen? |
| 02 | Gibt es konkrete Anwendungsfälle in Modernisierungsprogrammen, die passen könnten? |
| 03 | Seht ihr ein SaaS-Produkt in dem Konzept oder eher ein Werkzeug innerhalb von Modernisierungsprogrammen? |
| 04 | Gibt es eine konkrete interne Vorstellung eines neuen Portfolioelements? |

Rückfragen jederzeit direkt an mich, auch die unbequemen. Genau die haben bei 1.0 gefehlt.